

Spécification et preuve de programmes

Solution du devoir 2 2025-26

Alain Giorgetti

Master d'Informatique de l'Université Marie et Louis Pasteur

1 Table de vérité (1 point)

 Proposer et prouver un but ou un lemme WhyML qui permet de vérifier que le connecteur logique `&&` du langage WhyML, vu comme une fonction sur le type `bool`, est conforme à la table de vérité du “et” logique.

(1 point) Le but suivant ne convient pas car le symbole `=` est prioritaire par rapport au symbole `&&`, donc, par exemple, `False && False = False` correspond à `False && (False = False)`, qui se simplifie successivement en `False && True` et `False`.

```
goal andCheckWrong : False && False = False ∧ False && True = False
  ∧ True && False = False ∧ True && True = True
```

En ajoutant des parenthèses, on peut écrire le lemme suivant, qui correspond à l'énoncé, est vrai et est prouvé automatiquement par Why3.

```
lemma andCheck : (False && False) = False ∧ (False && True) = False
  ∧ (True && False) = False ∧ (True && True) = True
```

2 Différences entre deux tableaux (14 points)

Les réponses aux questions  sont à donner dans le fichier `devoir2spp.mlw`.

1. Expliquer par des phrases claires et simples ce que fait la fonction `search`, définie comme suit :

```
use int.Int
use ref.Ref
use array.Array

let search (a : array int) (x : int) : int =
  let ref i = 0 in
  let ref index = -1 in
  while i < a.length && index = -1 do
    if a[i] = x then index := i;
    i := i+1
  done;
  index
```

(0,5 points) L'appel de fonction `search a x` recherche l'entier `x` dans le tableau d'entiers `a`. Si l'entier `x` est dans le tableau, la fonction retourne l'indice d'un élément du tableau égal à `x`. Sinon, la fonction retourne -1.

2.  Spécifier cette fonctionnalité en WhyML, le plus complètement possible, par la précondition la plus générale possible et la postcondition la plus précise possible.

(1,5 points : 0,5 point par postcondition)

```
ensures { -1 ≤ result < a.length }
ensures { result = -1 → forall j. 0 ≤ j < a.length → a[j] ≠ x }
ensures { result ≠ -1 → a[result] = x }
```

La postcondition

```
ensures { result ≠ -1 → forall j. 0 ≤ j < result → a[j] ≠ x }
```

peut être ajoutée et un invariant plus précis que les suivants doit être ajouté pour la démontrer.

3.  Proposer des invariants et un variant de boucle WhyML pour démontrer automatiquement ce contrat et la terminaison de cette fonction `search`.

(2 points : 0,5 point par ligne)

```
invariant { 0 ≤ i ≤ a.length ∧ -1 ≤ index < a.length }
invariant { index = -1 → forall j. 0 ≤ j < i → a[j] ≠ x }
invariant { index ≠ -1 → a[index] = x }
variant { a.length - i }
```

4. La fonction du listing 1 stocke au début du tableau `c` les éléments présents dans le tableau `a` qui ne sont pas dans le tableau `b`. Cette fonction retourne aussi le nombre d'éléments ainsi stockés dans le tableau `c`.

```
let minus (a b c : array int) : int =
  let ref i = 0 in
  for j = 0 to a.length-1 do
    if search b a[j] = -1 then begin
      c[i] ← a[j];
      i := i+1
    end
  end
done;
i
```

Listing 1 – Fonction `minus`.

Donner un exemple de tableaux `a` et `b` avec au moins quatre éléments, tel que l'entier retourné par `(minus a b c)` soit supérieur ou égal à 2. Donner aussi le contenu du tableau `c` après cette exécution, dans cet exemple.

(1 point) Si $a = \{18, 3, 39, 50\}$ et $b = \{39, 18, 1, 42\}$, alors le résultat de $(\text{minus } a \ b \ c)$ est 2 et le début du tableau c est $\{3, 50\}$ après cette exécution.

5.  Proposer un contrat WhyML pour cette fonction `minus`, avec la précondition la plus générale possible et la postcondition la plus précise possible.

(2 points : 0,5 point par clause)

```
requires { c.length ≥ a.length } (* 0.5 pt *)
ensures { 0 ≤ result ≤ a.length } (* 0.5 pt *)
ensures { forall k. 0 ≤ k < result →
  (exists p. 0 ≤ p < a.length ∧ c[k] = a[p]) ∧
  (forall p. 0 ≤ p < b.length → c[k] ≠ b[p]) } (* 0.5 pt *)
ensures { forall k. 0 ≤ k < a.length →
  (not (exists p. 0 ≤ p < b.length ∧ b[p] = a[k])) →
  (exists p. 0 ≤ p < result ∧ c[p] = a[k]) } (* 0.5 pt *)
```

On peut y ajouter une postcondition sur la partie $c[\text{result}..]$ du tableau qui n'est pas modifiée. Aucun point ne porte sur cette postcondition.

6.  Proposer des invariants pour la boucle de la fonction `minus`, pour pouvoir démontrer automatiquement la correction de cette fonction, selon son contrat donné en réponse à la question 5.

(1,5 points : 0,5 point par invariant)

```
invariant { 0 ≤ i ≤ j ≤ a.length } (* 0.5 pt *)
invariant { forall k. 0 ≤ k < i →
  (exists p. 0 ≤ p < a.length ∧ c[k] = a[p]) ∧
  not (exists p. 0 ≤ p < b.length ∧ c[k] = b[p]) } (* 0.5 pt *)
invariant { forall k. 0 ≤ k < j →
  (not (exists p. 0 ≤ p < b.length ∧ b[p] = a[k])) →
  (exists p. 0 ≤ p < i ∧ c[p] = a[k]) } (* 0.5 pt *)
```

7.  Reproduire dans le fichier `devoir2spp.mlw` la fonction WhyML suivante, qui stocke au début du tableau c la différence symétrique des deux tableaux d'entiers a et b , composée des éléments présents dans le tableau a ou dans le tableau b , mais pas dans ces deux tableaux. Cette méthode retourne aussi le nombre d'éléments ainsi stockés dans le tableau c .

```
let diff (a b c : array int) : int =
  let ref i = 0 in
  for j = 0 to a.length-1 do
    if search b a[j] = -1 then begin
      c[i] ← a[j];
      i := i+1
    end
  done;
  let ref m = i in
  for j = 0 to b.length-1 do
    if search a b[j] = -1 then begin
      c[m] ← b[j];

```

```

    m := m+1
  end
done;
m

```

Par exemple, si $a = \{21, 12, 3, 7, 42, 42, 43, 50, 1\}$ et $b = \{1, 8, 42, 50, 24, 5\}$ avant exécution de l'instruction

```
let k = diff a b c in ..
```

et si c est un tableau d'entiers pré-alloué et de taille suffisante, alors on doit avoir $k = 8$ et $\{21, 12, 3, 7, 43, 8, 24, 5\}$ au début du tableau c après l'exécution de cette instruction.

Proposer un contrat WhyML pour cette fonction, avec la précondition la plus générale possible et la postcondition la plus précise possible.

(2,5 points : 0,5 point par clause)

```

requires { c.length ≥ a.length + b.length }      (* Q1: 0.5pt *)
ensures { 0 ≤ result ≤ a.length+b.length }      (* Q1: 0.5pt *)
ensures { forall k. 0 ≤ k < result →
  (exists p. 0 ≤ p < a.length ∧ c[k] = a[p]) ∧
  not (exists p. 0 ≤ p < b.length ∧ c[k] = b[p]) ∨
  (exists p. 0 ≤ p < b.length ∧ c[k] = b[p]) ∧
  not (exists p. 0 ≤ p < a.length ∧ c[k] = a[p]) } (* Q1: 0.5pt *)
ensures { forall k. 0 ≤ k < a.length →
  not (exists p. 0 ≤ p < b.length ∧ b[p] = a[k]) →
  (exists p. 0 ≤ p < result ∧ c[p] = a[k]) }      (* Q1: 0.5pt *)
ensures { forall k. 0 ≤ k < b.length →
  not (exists p. 0 ≤ p < a.length ∧ a[p] = b[k]) →
  (exists p. 0 ≤ p < result ∧ c[p] = b[k]) }      (* Q1: 0.5pt *)

```

La postcondition $0 \leq \text{result} \leq c.\text{length}$ est moins précise.

8.  Proposer des invariants pour les deux boucles de la fonction `diff`, pour pouvoir démontrer automatiquement la correction de cette fonction, selon son contrat donné en réponse à la question 7.

(1,5 point) Invariants pour la première boucle :

```

let ref i = 0 in
for j = 0 to a.length-1 do
  invariant { 0 ≤ i ≤ j ≤ a.length }              (* Q8: 0.5 pt *)
  invariant { forall k. 0 ≤ k < i →
    (exists p. 0 ≤ p < a.length ∧ c[k] = a[p]) ∧
    not (exists p. 0 ≤ p < b.length ∧ c[k] = b[p]) } (* Q8: 0.5 pt *)
  invariant { forall k. 0 ≤ k < j →
    not (exists p. 0 ≤ p < b.length ∧ b[p] = a[k]) →
    (exists p. 0 ≤ p < i ∧ c[p] = a[k]) }          (* Q8: 0.5 pt *)
  if search b a[j] = -1 then begin
    c[i] ← a[j];
    i := i+1
  end
end
done;

```

Les deux derniers invariants peuvent être remplacés par les suivants, plus précis :

```
invariant { forall k. 0 ≤ k < i →
  (exists p. 0 ≤ p < j ∧ c[k] = a[p]) ∧
  not (exists p. 0 ≤ p < b.length ∧ c[k] = b[p]) }
invariant { forall k. 0 ≤ k < j →
  not (exists p. 0 ≤ p < b.length ∧ b[p] = a[k]) →
  (exists p. 0 ≤ p < i ∧ c[p] = a[k]) }
```

(1,5 points) Invariants pour la deuxième boucle :

```
let ref m = i in
for j = 0 to b.length-1 do
  invariant { 0 ≤ i ≤ a.length } (* Q8: 0.25 pt *)
  invariant { 0 ≤ m-i ≤ j ≤ b.length } (* Q8: 0.25 pt *)
  invariant { forall k. 0 ≤ k < i →
    (exists p. 0 ≤ p < a.length ∧ c[k] = a[p]) ∧
    not (exists p. 0 ≤ p < b.length ∧ c[k] = b[p]) } (* Q8: 0.25 pt *)
  invariant { forall k. 0 ≤ k < a.length →
    not (exists p. 0 ≤ p < b.length ∧ b[p] = a[k]) →
    (exists p. 0 ≤ p < i ∧ c[p] = a[k]) } (* Q8: 0.25 pt *)
  invariant { forall k. i ≤ k < m →
    (exists p. 0 ≤ p < b.length ∧ c[k] = b[p]) ∧
    not (exists p. 0 ≤ p < a.length ∧ c[k] = a[p]) } (* Q8: 0.25 pt *)
  invariant { forall k. 0 ≤ k < j →
    not (exists p. 0 ≤ p < a.length ∧ a[p] = b[k]) →
    (exists p. i ≤ p < m ∧ c[p] = b[k]) } (* Q8: 0.25 pt *)
  if search a b[j] = -1 then begin
    c[m] ← b[j];
    m := m+1
  end
done;
```

3 Longueur d'une concaténation de deux listes, avec Why3 (5 points)

Cet exercice porte sur la définition du type inductif des listes en WhyML, introduite dans le cours 4 sur les types, et sur les implémentations en WhyML des fonctions *length* et *append* introduites dans le cours 8 sur les calculs inductifs. L'exercice doit être traité sans utiliser ni consulter le contenu du fichier `list.mlw` de la bibliothèque standard de Why3.

1. Dans le fichier `devoir2spp.mlw`, reproduire la définition du type inductif des listes du cours 4 et les définitions WhyML des fonctions *length* et *append* du cours 8.

(1 point)

```
use int.Int

type list 'a = Nil | Cons 'a (list 'a)

let rec function length (l : list 'a) : int
= match l with
  | Nil → 0
  | Cons _ l → 1 + length l
end

let rec function append (l m : list 'a) : list 'a
= match l with
  | Nil → m
  | Cons x l → Cons x (append l m)
end
```

2. Exprimer la propriété suivante sous la forme d'un but WhyML.

$$\text{length}(\text{append}(l, m)) = \text{length}(l) + \text{length}(m) \quad (1)$$

(1 point)

```
goal notAutomaticallyProved: forall l m : list 'a.
  length (append l m) = length l + length m
```

3. Tenter la preuve de ce but avec l'interface en ligne de Why3 et expliquer le résultat obtenu.

(1 point) Ce but n'est pas prouvé automatiquement car l'outil a besoin qu'on lui indique de réaliser la preuve par induction sur la liste l .

4.  Définir une fonction-lemme WhyML qui permet d'obtenir une preuve automatique de cette propriété avec l'interface en ligne de Why3.

(1 point)

```
let rec lemma lf1 (l m : list 'a): unit
  ensures { length (append l m) = length l + length m }
= match l with
| Nil → ()
| Cons _ r → lf1 r m
end
```

5.  Dans cette question, on suppose qu'il n'est pas possible d'écrire une fonction-lemme en WhyML. Proposer une postcondition pour la fonction `length` ou la fonction `append`, qui code la propriété (1) et dont la preuve est automatique avec l'interface en ligne de Why3.

(1 point)

```
let rec function appendPropLength (l m : list 'a) : list 'a
  ensures { length result = length l + length m }
= match l with
| Nil → m
| Cons x l → Cons x (append l m)
end
```